SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME:Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

ANSWERS

1. ATM Withdrawal Module (Black-Box Reverse Engineering)

Since only input-output behaviour of the ATM withdrawal module is known, the first step is to infer the hidden specification from the observations. The observed behaviour suggests the following rules: the amount must be a positive multiple of 100, a single transaction cannot exceed a per-transaction limit (assumed Rs. 20,000), the amount cannot exceed the available balance, a minimum balance (assumed Rs. 500) must remain after withdrawal, and a daily limit (assumed Rs. 40,000) applies. Each violated rule produces a distinct error message such as "Invalid amount", "Limit exceeded" or "Insufficient balance". The module is therefore decomposed into five functional components: card and PIN authentication, amount input validation, limit checking, balance validation, and cash dispensing with balance update and receipt generation.

From these components, the equivalence classes are identified. For the amount field, the valid class is a multiple of 100 between 100 and 20,000, while the invalid classes are zero or negative values, non-numeric input, values that are not multiples of 100, and values above 20,000. For the balance condition, the valid class is amount less than or equal to (balance minus 500), and the invalid class is amount greater than that value. The boundary conditions are 0, 99, 100, 101, 19,900, 20,000 and 20,100 for the amount, and exact-balance, balance minus one and balance plus one for balance validation. The observed input-output pairs indicate that the original testers mainly used Equivalence Class Partitioning with a few random boundary checks, together with error guessing for the error messages.

The adequacy of this approach is evaluated as partial. Boundary values on the lower side (0, 99, 100) and the daily-limit boundary were not examined, combinations of two violated rules (for example, an amount above the limit and above the balance at the same time) were not tested to see which message takes priority, and repeated withdrawals in one day were not covered. An improved suite is therefore reconstructed: TC1 withdraw 100 (minimum valid) and expect success; TC2 withdraw 20,000 with sufficient balance and expect success; TC3 withdraw 20,100 and expect "Limit exceeded"; TC4 withdraw 99 and 150 and expect "Invalid amount"; TC5 withdraw 0, a negative value and a letter and expect "Invalid amount"; TC6 withdraw exactly (balance minus 500) and expect success; TC7 withdraw (balance minus 499) and expect "Insufficient balance"; TC8 withdraw 25,000 with a balance of 10,000 and verify the error priority; TC9 perform two withdrawals totalling above 40,000 and expect the daily-limit error; and TC10 enter a wrong PIN three times and expect the card to be blocked. This set covers every equivalence class, every boundary and the critical rule interactions, and it is documented in a table of test case ID, input, expected output and technique applied (ECP, BVA or error guessing).

2. Execution Traces and Code Snippets (White-Box Reverse Engineering)

Without any test documentation, the white-box approach is recovered from the code logic itself. The control flow graph (CFG) is constructed by converting each statement block into a node and each possible transfer of control into an edge. Consider a representative module that reads an input n, initialises sum to zero, executes a loop while i is less than or equal to n, inside which an if-else condition checks whether i is even, and finally checks whether sum is greater than a threshold before returning a result. The nodes are: N1 (initialisation), N2 (loop condition), N3 (even check), N4 (even branch), N5 (odd branch), N6 (increment and return to the loop), N7 (final threshold decision), N8 and N9 (the two outcomes) and N10 (exit). The decision points are N2, N3 and N7, and there is one loop.

The cyclomatic complexity is calculated as V(G) = E - N + 2P. For the graph above with 12 edges, 10 nodes and one connected component, V(G) = 12 - 10 + 2 = 4, which is also equal to the number of decision nodes plus one. Therefore four independent paths exist: P1 skips the loop entirely and takes the threshold decision to its first outcome; P2 enters the loop, takes the even branch and exits with the second outcome; P3 enters the loop, takes the odd branch and exits; and P4 iterates the loop more than once with both branches taken. The execution traces provided are mapped onto these paths and they are found to repeat P2 and P3 only, so only statement coverage and part of the branch coverage are achieved. The loop-skip path, the multiple-iteration path and the false outcome of the final decision are never executed.

The reconstructed path testing strategy is basis-path testing combined with loop testing. One test case is designed for each independent path: n = 0 for P1, n = 1 for the odd branch, n = 2 for the even branch, and n = 5 (several iterations) for P4, with a threshold value chosen so that both outcomes of the final decision occur. Loop testing adds zero, one, two, a typical number and the maximum number of iterations. With these tests, 100% statement, branch and independent path coverage are obtained, and the findings are presented with the CFG, the V(G) calculation and a path-versus-test-case mapping table.

3. Web Application Login and Transaction System (Defect-Log Analysis)

By analysing the defect logs, the testing technique likely used earlier can be inferred from the pattern of defects found and missed. Defects such as "valid user cannot log in" and "empty password accepted" indicate that basic equivalence partitioning of valid and invalid inputs was performed. Defects around the account lockout after exactly three attempts, a password of exactly the minimum or maximum length and a transaction exactly at the daily limit indicate that some boundary value analysis was done. Defects in the transaction flow, such as a failed debit still recording a credit or a session timeout in the middle of payment, point to a few ad hoc path-based or error-guessing checks, while no defects at all are reported for rarely used branches, which suggests that structured white-box testing was absent.

The system is decomposed into testable components: the login module (username and password validation, captcha, lockout, session creation), the transaction module (amount validation, balance check, debit and credit processing, confirmation), the session and security component, and the database layer. The coverage gaps identified are: no systematic negative testing of special characters and SQL injection in login fields, no boundary tests for the minimum and maximum transaction amount, no testing of the combinations of conditions in the transaction approval logic, no loop coverage for retry attempts, and no concurrency or timeout tests.

The reconstructed strategy integrates three techniques. Equivalence partitioning is applied to the username, password and amount fields, producing valid and invalid classes. Boundary value analysis is applied to the password length (for example 7, 8, 16 and 17 characters), the number of login attempts (2, 3 and 4), and the transaction amount (0, 1, limit and limit plus 1). Path testing is applied by drawing the CFG of the login and the transaction logic and executing every independent path, including the failed-authentication path, the lockout path, the insufficient-funds path and the rollback path. This combination is justified because black-box methods find specification and input-handling faults, whereas path testing exposes logic faults that are invisible from the outside; together they close the gaps that the defect log reveals and reduce the probability of residual defects.

4. Legacy Inventory Management System (Log-Based Black-Box Design)

The transaction logs of successful and failed stock updates are analysed to infer the hidden business rules. Successful logs show that stock increases with a positive receipt quantity and decreases with a positive issue quantity as long as the result is not negative; failed logs show rejections for a zero or negative quantity, a quantity larger than the stock on hand, and an unknown item code. The low-stock alert appears in the log when the stock after the update falls to or below a reorder level (for example 10 units) and not when it is above it. From these observations, the hidden boundary conditions are the quantity limits (0 and 1), the stock limit (issue quantity equal to stock and stock plus one), and the reorder level (reorder level plus one, reorder level, and reorder level minus one).

The update process is decomposed into functional units: item validation, quantity validation, stock availability check, stock update, and alert generation. The equivalence partitions derived are: valid item code and invalid item code; quantity less than or equal to zero, quantity between one and the available stock, and quantity greater than the available stock; and resulting stock above, equal to, or below the reorder level. The logs show that only the typical values in the valid partitions and a few large invalid quantities were tested, so the boundary at which the alert is triggered and the exact-stock issue (resulting stock of zero) were never exercised.

The reconstructed black-box design therefore contains the following test cases: issue of quantity 0 and -1 (rejected); issue of quantity 1 (accepted); issue of quantity equal to the stock (accepted, stock becomes zero, alert raised); issue of stock plus one (rejected); an issue leaving the stock at reorder level plus one (no alert);

an issue leaving the stock at the reorder level (alert raised); an issue leaving the stock at reorder level minus one (alert raised); receipt of a positive quantity that raises the stock above the reorder level (alert cleared); and an update with an unknown item code (rejected). Each test case is selected because it lies on a boundary or represents a distinct partition inferred from the logs, which guarantees that all hidden rules are verified with a minimum number of tests.

5. Library Management Executable with CPU/Memory Profile (Conceptual CFG)

The CPU and memory profile of the stress test is used as indirect evidence of the internal structure of the executable. Sharp, periodic CPU spikes that grow with the number of records indicate loops whose iteration count depends on the input size, such as a loop that searches the catalogue for a book or a loop that scans the borrower list to calculate fines. A steady memory increase that is not released after the operation indicates a loop or recursion that allocates objects, for example building a result list for each search request. Flat segments with small spikes correspond to simple decision clusters, such as checking membership validity, book availability and the maximum number of issued books.

From this mapping, a conceptual CFG is constructed. It begins with the Start node, followed by a decision on the validity of the member, a decision on the availability of the book, a loop node (search through catalogue records), a nested decision inside this loop (match found or not), a second loop (calculation of fine over the overdue records), a decision for the maximum limit of books, and an End node. The structure has four decisions and two loops, so the estimated cyclomatic complexity is approximately 7, and the nested loop region is where the CPU usage is highest. Because the source code is unavailable, the graph is conceptual and its nodes are labelled by behaviour rather than by statement.

The path testing strategy targets the resource-intensive paths first: the path through the search loop with zero matches (which runs the full loop), the path with the maximum number of records, the nested loop path with many overdue records, and the path that repeats the issue and return operations continuously to expose memory leaks. Loop tests with zero, one, typical and maximum iterations are included. The impact on system stability is evaluated by running these paths under load and observing whether the response time, CPU usage and memory consumption stay within acceptable limits; any growth in memory after the test shows a leak, and any non-linear CPU increase shows an inefficient loop that must be optimised. In this way, the testing focuses effort on the paths most likely to cause crashes or slowdowns and improves the stability of the system.

6. Mobile Payment Gateway (State Transition Testing)

By analysing the JSON responses, the gateway is modelled as a finite state machine. The states are Initiated (the start state), Pending, Success, Reversal and Timeout, and the triggers are payment submission, bank confirmation, bank failure or rejection, no response within the time limit, and a reversal request. The data constraints observed in the JSON fields are that a transaction ID remains the same through all states, the

amount in a Reversal must be equal to or less than the amount of the original Success, and the status code changes with each state. The legal transitions inferred are: Initiated to Pending on submission; Pending to Success on bank confirmation; Pending to Timeout when no response arrives within the timeout; Timeout to Success or Failed when the delayed bank response arrives; and Success to Reversal when a valid reversal request is made. The illegal transitions are those such as Reversal to Success, Success to Pending, Pending to Reversal (since nothing has yet been debited), and a second Reversal of the same transaction.

The technique implicitly used by the original designers is State Transition Testing, because each response depends on the current state and the event that occurred, not simply on the input value. A transition table with the current state, event, next state and expected response is the structured form of this model, and a state transition diagram would show the circles Initiated, Pending, Success, Reversal and Timeout joined by arrows labelled with the triggering events, with Success and Reversal drawn as the final states.

The reconstructed test suite covers all legal transitions (0-switch coverage) and all illegal transitions. The legal tests are: submit payment and confirm it; submit payment and let it time out; receive a late confirmation after the timeout; and reverse a successful payment with the full amount and with a partial amount. The illegal tests are: reversing a pending transaction, reversing a transaction twice, reversing with an amount larger than the original, confirming a reversed transaction, and sending a confirmation for a non-existent transaction ID; each must be rejected with an appropriate error response and no change in state. Sequences of two consecutive transitions (1-switch coverage) such as Pending, Timeout, Success are also included to detect faults in state history handling. This ensures every state, transition and invalid event of the gateway is verified.

7. Encrypted Communication Module (Fuzzing and BVA)

Only the inputs and the encrypted outputs of the module are visible, so its validation logic is reverse engineered by fuzzing. Large numbers of malformed inputs are sent to each field, for example empty values, very long strings, special characters, null bytes, format strings, SQL and script fragments, and unexpected data types, and the response to each is recorded. The inputs that are rejected with a specific error code reveal the validation boundaries, for instance a message length limit of 256 characters, a key length that must be exactly 16, 24 or 32 characters, and an allowed character set for the recipient field. The inputs that cause a crash, a hang or an unexpected output reveal weaknesses in error handling.

The module is decomposed into two components. The data validation component checks the length, type, format and character set of the message, key and recipient fields before processing, and the encryption component takes only validated data and generates the ciphertext. Separating them is important because every fault found by fuzzing must be traced to a component: a rejected input points to validation, while a malformed or non-deterministic output points to encryption.

The improved test cases are constructed using Boundary Value Analysis on each field. For the message length of 1 to 256, the tests are 0, 1, 2, 255, 256 and 257 characters; for the key length, the tests are 15, 16, 17, 31, 32 and 33 characters; and for the recipient field, the tests are minimum, maximum and maximum plus one length. Each boundary test is combined with special characters, escape sequences and typical injection strings. These tests prevent buffer overflow because the input of maximum length plus one proves that the module rejects or truncates the data instead of writing beyond the allocated buffer, and they prevent injection because the tests with malicious payloads show that all data is validated and treated as data rather than being executed. The tests also confirm that the error messages do not reveal internal information, and that decrypting the ciphertext returns the original message for every valid boundary value

8. Insurance Premium Calculator (Condition Coverage Inadequacy)

A result of 100% statement coverage means only that each statement was executed at least once; it says nothing about whether each outcome of every decision was exercised. The failure to detect defects in the multi-condition if statements shows that the legacy tests were designed using statement coverage alone, with tests that typically satisfied the conditions in the combination that executes the body. For example, in the statement if (age > 60 AND smoker == true OR claims > 2) premium = premium * 1.5, a single test with age 65 and smoker true executes the statement and gives full statement coverage, but the false outcomes and the individual contribution of each condition are never tested, so a wrong operator or a wrong constant stays hidden.

A new CFG is constructed for the calculator with one node for reading the inputs, one decision node for each atomic condition (age check, smoker check, claims check), the node for premium loading, further decision nodes for the discount or vehicle/health category, and the exit node. Because compound conditions are split into separate decision nodes, the graph has, for instance, 6 decision nodes and a cyclomatic complexity V(G) = 7, so seven independent paths must be tested, such as: no condition true; only age true; age and smoker true; only claims true; both parts true; discount path; and loading plus discount path.

The reconstructed strategy is to apply branch coverage (each decision true and false), condition coverage (each atomic condition true and false) and basis-path coverage for the seven independent paths. Test data are chosen on boundaries, such as age 60 and 61 and claims 2 and 3, with smoker true and false in combination. A test suite of about seven to eight cases then yields 100% branch and path coverage. This reduces the risk of calculation errors because every condition is shown to influence the outcome independently, wrong relational operators (for example > in place of >=), wrong logical operators (AND in place of OR) and missing branches are exposed, and each premium rate is verified against its expected value.

9. Cloud File Synchronization Tool (ECP, BVA and Stress Testing)

The lost-file defect reports are analysed by grouping them by file size, file type and network speed. The pattern found is that files are lost mainly when the size is above a certain threshold (for example above 2 GB), when

the file type is unusual or has no extension, when the file name contains special characters, and when the network speed is low or unstable so that the upload is interrupted. Few or no defects are found for small files of common types over a fast network. This suggests that the synchronization algorithm uploads in chunks, with the size, the file type filter and the network connection as the deciding variables, and that the sync marks the file as completed before the checksum of the uploaded data is verified.

The Equivalence Class Partitioning likely used by the developers is therefore: file size as small (up to 1 MB), medium (1 MB to 100 MB), large (100 MB to 2 GB) and very large (above 2 GB); file type as common documents, media files, executables or archives, and unsupported or zero-byte files; and network speed as fast, slow and intermittent or disconnected. The gap is that the developers tested a representative value in each class but did not test at the class limits or with combinations such as a large file over a slow network.

The revised strategy integrates Boundary Value Analysis with stress testing. Boundary values include a zero-byte file, 1 byte, 1 MB minus and plus one byte, and 2 GB minus, equal to and plus one byte; the network is varied between normal, throttled and dropped mid-transfer. Stress testing then runs these cases with many files, large simultaneous uploads from several devices, low disk space and repeated disconnections, to check how the tool behaves under extreme load. The expected results include a complete resume of an interrupted upload, matching checksums of source and destination, and a clear error message, with no silent deletion. This approach prevents future data loss because it exercises exactly the conditions in which files were lost, verifies integrity before a file is marked as synchronized, and detects faults in retry and conflict-handling logic before release.

10. E-Commerce Search and Filter Module (Decision Table Testing)

The SQL queries show how each selected filter becomes a condition in the WHERE clause. Filters from different groups (for example category, brand, price range, rating and availability) are joined by AND, several values within the same group (for example Brand = A or Brand = B) are joined by OR, and the exclusion option (for example not out-of-stock, or not a particular brand) appears as NOT. Empty filters are omitted from the query so that no restriction is applied, and the sort order or keyword adds a further condition. The decision-making logic is therefore of the form: (Category = c) AND (Brand = b1 OR Brand = b2) AND (Price BETWEEN min AND max) AND (Rating >= r) AND NOT (OutOfStock).

The reconstructed decision-table based strategy treats each filter as a condition with the values selected or not selected, and the resulting dataset behaviour as the action: return the matching products, return all products, or return an empty result with a "no results" message. For example, with four conditions (category selected, brand selected, price range valid and availability filter on), there are 2 raised to 4, that is 16 combinations, and each column of the table represents one rule with its expected result. Rules that are impossible or duplicate, such as a minimum price greater than the maximum price, are marked as invalid and expected to produce a validation error, and rules with identical actions are merged to reduce the table. Boundary values for price

(equal to minimum and equal to maximum) and for rating are additionally tested within the rules that include these conditions.

The completeness of the model is evaluated by checking that every combination of the conditions appears in exactly one rule, that every rule has a defined action, and that every filter in the observed SQL queries is represented in the table. The model is complete for the filters visible in the logged queries, but it is limited in two ways: the hidden filters that did not appear in the queries cannot be modelled, and the interaction with sorting, pagination and keyword search increases the combinations, so these are handled by a separate decision table or by pairwise reduction. With these limits stated, the decision table gives a systematic, traceable test set covering every search combination that the module supports.

Code of Conduct:

I R.RACHITHA.R/192421058 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation